

CHAMPIONTRACKPRO V2 — PRD CORRIGÉ

Lu conjointement avec CLAUDE_CODE_CONSIGNES.md

CONTEXTE ARCHITECTURAL CRITIQUE

Tu travailles sur une base de code V1 existante avec des données réelles.

Avant toute modification, lis :

- firestore.rules (règles actuelles)
- La structure actuelle : teams/{teamId}/trainings/{trainingId}/responses/{uid}
- src/screens/PerformanceDashboard.tsx (dashboard existant)
- functions/index.js (Cloud Functions existantes)

NE PAS créer de nouvelles collections sans migrer ou mapper l'existant.

NE PAS casser la structure teams/{teamId}/trainings/{trainingId}.

PILIERES STRATÉGIQUES

1. Zero Medical Liability : on track "Wellness" uniquement — jamais douleur, jamais clinique
 2. Coach Full Access : accès complet aux données de son équipe uniquement
 3. UX < 60 secondes (réaliste — pas 15s)
 4. Decision Support : Readiness Score + EMA remplacent les graphes bruts
-

PHASE 1 — AUDIT

Audite ces fichiers spécifiquement :

- firestore.rules → cherche allow read: if true, IDOR, accès cross-team

- src/screens/PerformanceDashboard.tsx → filtrage côté client vs serveur
- functions/index.js → données orphelines, sécurité

Génère un rapport AUDIT_REPORT.md avec :

-  Ce qui est correct
-  Vulnérabilités critiques + fichier + ligne exacte
-  Dettes techniques
-  Plan de remédiation priorisé

PHASE 2 — SCHÉMA BASE DE DONNÉES

Structure à conserver (V1 existante)

```
teams/{teamId}/trainings/{trainingId}    → séance
teams/{teamId}/trainings/{trainingId}/responses/{uid} → réponse athlète
users/{uid}                               → PII + fcmWebTokens
```

Champs à ajouter dans responses/{uid}

```
{
  // Existant – NE PAS SUPPRIMER
  uid: string,
  trainingId: string,
  teamId: string,
  submittedAt: Timestamp,
  isTest: boolean,

  // NOUVEAU V2 – métriques questionnaire
  metrics: {
    cardioLoad:      number, // 1-10, élevé = mauvais
    neuroLoad:       number, // 1-10, élevé = mauvais
    sleepQuality:    number, // 1-10, élevé = mauvais ← ATT
```

ENTION inversé vs PRD original

```
    stressLevel:      number,    // 1-10, élevé = mauvais
    motorControl:     number,    // 1-10, élevé = mauvais
    tacticalLucidity: number | null, // null si sessionType
    != scrimmage
    sessionRPE:       number,    // 1-10 – charge globale ressentie
  },

  // NOUVEAU V2 – calculé
  readinessScore: number,        // 0-100
  workloadAU:      number,        // sessionRPE × durée en minutes

  // NOUVEAU V2 – contexte
  sessionType: "conditioning" | "skill" | "scrimmage",
}
```

PAS de collection daily_logs

Toutes les données restent dans teams/{teamId}/trainings/{trainingId}/responses/{uid}.
Pas de duplication de PII.

Friction Matrix – VERSION LÉGALE UNIQUEMENT

Remplace "Physical Soreness" par "Physical Fatigue" pour éviter toute interprétation médicale.

```
frictionType: "Physical Fatigue" | "Academic/Life Stress" |
              "Court Confusion" | "Mental/Emotional"
// PAS de "Pain", "Injury", "Soreness" – risque HIPAA
```

Règles Firestore

```
// Athlète : lit/écrit uniquement ses propres réponses dans son équipe
```

```

match /teams/{teamId}/trainings/{trainingId}/responses/{uid}
{
  allow read, write: if request.auth.uid == uid
    && getUserData().teamId == teamId;
}

// Coach : lit toutes les réponses de son équipe uniquement
match /teams/{teamId}/trainings/{trainingId}/responses/{uid}
{
  allow read: if isCoachOfTeam(teamId);
}

// Pas de collection ai_training_dataset accessible client
match /ai_training_dataset/{doc} {
  allow read, write: if false;
}

```

PHASE 3 – QUESTIONNAIRE ATHLÈTE

Composant : src/screens/StitchQuestionnaireScreen.js (modifier l'existant)

Ne pas créer un nouveau fichier — modifier le questionnaire existant.

SemanticSlider — règles strictes

- Zéro symbole mathématique (+/-)
- Zéro chiffre visible au repos
- Tooltip flottant avec valeur 1-10 UNIQUEMENT pendant le drag
- Anchres sémantiques gauche/droite en rgba(255,255,255,0.38)
- Thumb : gradient cyan #00BFFF → #0066FF

Direction des sliders — TOUTES dans le même sens

Score élevé = mauvais/fatigue — SANS EXCEPTION pour éviter les bugs de normalisation.

cardioLoad :

Q: "How did your lungs handle the pace today?"

Left: Never out of breath Right: Completely Gassed

neuroLoad :

Q: "How bouncy and explosive did your legs feel?"

Left: Bouncy / Explosive Right: Heavy / Glued to the floor

sleepQuality :

Q: "How restorative was your sleep last night?"

Left: Deep / Woke up fresh Right: Terrible / Restless

← INVERSÉ par rapport au PRD original pour cohérence

stressLevel :

Q: "What's your current stress level? (school + hoops)"

Left: Completely relaxed Right: Overwhelmed

motorControl :

Q: "How did your shot and handle feel today?"

Left: Pure / Automatic Right: Clumsy / Bricking

tacticalLucidity (scrimmage uniquement) :

Q: "How well did you process defensive rotations?"

Left: Reading the floor Right: Lost / A step behind

sessionRPE (toujours, en dernier) :

Q: "Overall, how hard was today's session?"

Left: Active Recovery Right: Hardest session ever

Questions conditionnelles par sessionType

```
conditioning → cardioLoad, neuroLoad, sleepQuality, stressLevel, sessionRPE
skill → cardioLoad, neuroLoad, sleepQuality, stressLevel, motorControl, sessionRPE
scrimmage → toutes les 7 questions
```

Le coach définit sessionType quand il crée la séance dans le planning.

Friction Matrix

Affichée après les sliders si hasFriction === true :

- frictionType (select) : "Physical Fatigue" | "Academic/Life Stress" | "Court Confusion" | "Mental/Emotional"
- frictionFrequency (slider) : Rarely → Constantly
- frictionImpact (slider) : Barely noticeable → Severely limiting
- frictionDistraction (slider) : Not worried → Highly distracted

PHASE 4 — DASHBOARD COACH

Modifier src/screens/PerformanceDashboard.tsx — ne pas recréer.

Readiness Score (0-100)

```
// Toutes les métriques dans le même sens – élevé = mauvais
// Donc on inverse pour obtenir le score de forme
const calculateReadiness = (m: Metrics): number => {
  const scores = {
    cardio: (10 - m.cardioLoad) * 0.20,
    neuro: (10 - m.neuroLoad) * 0.25,
    sleep: (10 - m.sleepQuality) * 0.20,
    stress: (10 - m.stressLevel) * 0.15,
    motor: (10 - m.motorControl) * 0.10,
    tactical: (10 - (m.tacticalLucidity ?? m.stressLevel)) *
    0.10,
```

```

    }
    const weighted = Object.values(scores).reduce((a, b) => a +
b, 0)
    return Math.round((weighted / 10) * 100)
  }

```

EMA (Exponential Moving Average)

```

const N = 28
const alpha = 2 / (N + 1)

const calculateEMA = (values: (number | null)[]): number[] =>
{
  const ema: number[] = []
  values.forEach((v, i) => {
    if (i === 0) {
      ema.push(v ?? 5) // seed avec valeur neutre si null
    } else {
      const prev = ema[i - 1]
      ema.push(v !== null ? (v * alpha) + (prev * (1 - alph
a)) : prev)
    }
  })
  return ema
}

const calculateDeviation = (value: number, ema: number): numb
er => {
  return ((value - ema) / ema) * 100
}

```

Morning Brief — tri par risque

- Danger : readinessScore < 40 ou deviation > +20%
- Monitor : readinessScore 40-65 ou deviation +10% à +20%

● Optimal : readinessScore > 65 et deviation < +10%

Athlètes ● en premier, puis ●, puis ●.

Charts à implémenter (Recharts)

1. RadarChart — Physical / Mental / Technical
2. ComposedChart déviation — bars colorées + ligne EMA
3. ComposedChart workload — EMA 7j cyan + EMA 28j bleu dashed + zone danger rouge

PHASE 5 — AI DATA LAKE

Cloud Function : anonymizePlayerDataForAI

Trigger : suppression document users/{uid}

Logic :

1. Query teams/{teamId}/trainings/*/responses/{uid}
2. Strip uid, teamId, name, email
3. Inject position, sport: "basketball", ageAtLog
4. Write to ai_training_dataset via db.batch()
5. Hard delete responses originales

PHASE 6 — RAPPORT FINAL

Génère ChampionTrackPro_V2_Implementation_Report.md UNIQUEMENT quand je dis : "All phases complete, generate handover report."

ORDRE D'EXÉCUTION

Execute dans cet ordre sans t'arrêter entre les phases.

Auto-validation obligatoire après chaque phase (voir CLAUDE_CODE_CONSIGNES.md).

Commit après chaque phase avec message clair.

Phase 1 → Phase 2 → Phase 3 → Phase 4 → Phase 5

Ne demande pas de confirmation entre les phases.

Si tu bloques sur un point ambigu → prends la décision la plus conservative et documente-la dans un fichier DECISIONS.md à la racine.